```
int main()
{
   Compound u;
   u.x = 1000;
   cout << u.y << endl;
   return 0;
}
```

Programming Challenges

1. **Movie Data**

   Write a program that uses a structure named `MovieData` to store the following information about a movie:

   Title
   Director
   Year Released
   Running Time (in minutes)

   The program should create two `MovieData` variables, store values in their members, and pass each one, in turn, to a function that displays the information about the movie in a clearly formatted manner.

2. **Movie Profit**

   Modify the Movie Data program written for Programming Challenge 1 to include two additional members that hold the movie's production costs and first-year revenues. Modify the function that displays the movie data to display the title, director, release year, running time, and first year's profit or loss.

3. **Corporate Sales Data**

   Write a program that uses a structure to store the following data on a company division:

   Division Name (such as East, West, North, or South)
   First-Quarter Sales
   Second-Quarter Sales
   Third-Quarter Sales
   Fourth-Quarter Sales
   Total Annual Sales
   Average Quarterly Sales

   The program should use four variables of this structure. Each variable should represent one of the following corporate divisions: East, West, North, and South. The user should be asked for the four quarters' sales figures for each division. Each division's total and average sales should be calculated and stored in the appropriate member of each structure variable. These figures should then be displayed on the screen.

   Input Validation: Do not accept negative numbers for any sales figures.


VideoNote
Solving the Weather Statistics Problem

4. Weather Statistics

Write a program that uses a structure to store the following weather data for a particular month:

Total Rainfall
High Temperature
Low Temperature
Average Temperature

The program should have an array of 12 structures to hold weather data for an entire year. When the program runs, it should ask the user to enter data for each month. (The average temperature should be calculated.) Once the data are entered for all the months, the program should calculate and display the average monthly rainfall, the total rainfall for the year, the highest and lowest temperatures for the year (and the months they occurred in), and the average of all the monthly average temperatures.

Input Validation: Only accept temperatures within the range between –100 and +140 degrees Fahrenheit.

5. Weather Statistics Modification

Modify the program that you wrote for Programming Challenge 4 so it defines an enumerated data type with enumerators for the months (`JANUARY`, `FEBRUARY`, etc.). The program should use the enumerated type to step through the elements of the array.

6. Soccer Scores

Write a program that stores the following data about a soccer player in a structure:

Player's Name
Player's Number
Points Scored by Player

The program should keep an array of 12 of these structures. Each element is for a different player on a team. When the program runs it should ask the user to enter the data for each player. It should then show a table that lists each player's number, name, and points scored. The program should also calculate and display the total points earned by the team. The number and name of the player who has earned the most points should also be displayed.

Input Validation: Do not accept negative values for players' numbers or points scored.

7. Customer Accounts

Write a program that uses a structure to store the following data about a customer account:

Name
Address
City, State, and ZIP
Telephone Number
Account Balance
Date of Last Payment

The program should use an array of at least 10 structures. It should let the user enter data into the array, change the contents of any element, and display all the data stored in the array. The program should have a menu-driven user interface.

Input Validation: When the data for a new account is entered, be sure the user enters data for all the fields. No negative account balances should be entered.

8. **Search Function for Customer Accounts Program**

   Add a function to Programming Challenge 7 that allows the user to search the structure array for a particular customer's account. It should accept part of the customer's name as an argument and then search for an account with a name that matches it. All accounts that match should be displayed. If no account matches, a message saying so should be displayed.

9. **Speakers' Bureau**

   Write a program that keeps track of a speakers' bureau. The program should use a structure to store the following data about a speaker:

   Name
   Telephone Number
   Speaking Topic
   Fee Required

   The program should use an array of at least 10 structures. It should let the user enter data into the array, change the contents of any element, and display all the data stored in the array. The program should have a menu-driven user interface.

   Input Validation: When the data for a new speaker is entered, be sure the user enters data for all the fields. No negative amounts should be entered for a speaker's fee.

10. **Search Function for the Speakers' Bureau Program**

    Add a function to Programming Challenge 9 that allows the user to search for a speaker on a particular topic. It should accept a key word as an argument and then search the array for a structure with that key word in the Speaking Topic field. All structures that match should be displayed. If no structure matches, a message saying so should be displayed.

11. **Monthly Budget**

    A student has established the following monthly budget:

Housing	500.00
Utilities	150.00
Household Expenses	65.00
Transportation	50.00
Food	250.00
Medical	30.00
Insurance	100.00
Entertainment	150.00
Clothing	75.00
Miscellaneous	50.00

Write a program that has a `MonthlyBudget` structure designed to hold each of these expense categories. The program should pass the structure to a function that asks the user to enter the amounts spent in each budget category during a month. The program should then pass the structure to a function that displays a report indicating the amount over or under in each category, as well as the amount over or under for the entire monthly budget.

12. Course Grade

Write a program that uses a structure to store the following data:

Member Name	Description
Name	Student name
Idnum	Student ID number
Tests	Pointer to an array of test scores
Average	Average test score
Grade	Course grade

The program should keep a list of test scores for a group of students. It should ask the user how many test scores there are to be and how many students there are. It should then dynamically allocate an array of structures. Each structure's `Tests` member should point to a dynamically allocated array that will hold the test scores.

After the arrays have been dynamically allocated, the program should ask for the ID number and all the test scores for each student. The average test score should be calculated and stored in the `average` member of each structure. The course grade should be computed on the basis of the following grading scale:

Average Test Grade	Course Grade
91–100	A
81–90	B
71–80	C
61–70	D
60 or below	F

The course grade should then be stored in the `Grade` member of each structure. Once all this data is calculated, a table should be displayed on the screen listing each student's name, ID number, average test score, and course grade.

Input Validation: Be sure all the data for each student is entered. Do not accept negative numbers for any test score.

13. Drink Machine Simulator

Write a program that simulates a soft drink machine. The program should use a structure that stores the following data:

Drink Name
Drink Cost
Number of Drinks in Machine

The program should create an array of five structures. The elements should be initialized with the following data:

Drink Name	Cost	Number in Machine
Cola	.75	20
Root Beer	.75	20
Lemon-Lime	.75	20
Grape Soda	.80	20
Cream Soda	.80	20

Each time the program runs, it should enter a loop that performs the following steps: A list of drinks is displayed on the screen. The user should be allowed to either quit the program or pick a drink. If the user selects a drink, he or she will next enter the amount of money that is to be inserted into the drink machine. The program should display the amount of change that would be returned and subtract one from the number of that drink left in the machine. If the user selects a drink that has sold out, a message should be displayed. The loop then repeats. When the user chooses to quit the program it should display the total amount of money the machine earned.

Input Validation: When the user enters an amount of money, do not accept negative values or values greater than $1.00.

14. Inventory Bins

Write a program that simulates inventory bins in a warehouse. Each bin holds a number of the same type of parts. The program should use a structure that keeps the following data:

Description of the part kept in the bin
Number of parts in the bin

The program should have an array of 10 bins, initialized with the following data:

Part Description	Number of Parts in the Bin
Valve	10
Bearing	5
Bushing	15
Coupling	21
Flange	7
Gear	5
Gear Housing	5
Vacuum Gripper	25
Cable	18
Rod	12

The program should have the following functions:

AddParts: a function that increases a specific bin's part count by a specified number.

RemoveParts: a function that decreases a specific bin's part count by a specified number.

When the program runs, it should repeat a loop that performs the following steps: The user should see a list of what each bin holds and how many parts are in each bin. The user can choose to either quit the program or select a bin. When a bin is selected, the user can either add parts to it or remove parts from it. The loop then repeats, showing the updated bin data on the screen.

Input Validation: No bin can hold more than 30 parts, so don't let the user add more than a bin can hold. Also, don't accept negative values for the number of parts being added or removed.

15. Multipurpose Payroll

Write a program that calculates pay for either an hourly paid worker or a salaried worker. Hourly paid workers are paid their hourly pay rate times the number of hours worked. Salaried workers are paid their regular salary plus any bonus they may have earned. The program should declare two structures for the following data:

Hourly Paid:
HoursWorked
HourlyRate

Salaried:
Salary
Bonus

The program should also declare a `union` with two members. Each member should be a structure variable: one for the hourly paid worker and another for the salaried worker.

The program should ask the user whether he or she is calculating the pay for an hourly paid worker or a salaried worker. Regardless of which the user selects, the appropriate members of the `union` will be used to store the data that will be used to calculate the pay.

Input Validation: Do not accept negative numbers. Do not accept values greater than 80 for HoursWorked.